

Programming Using C++ Level 1

User inputs



Programming Using C++

Level 1

User inputs

Definition

- ◆ User input allows the user to enter values while the program is running instead of having the values already written in the code.
- ◆ In C++, the main tools used for user input are `cin` and `getline()`

Explanation

- ◆ First, we declare the variables without assigning values to them.

```
string Name;
```

```
int Age;
```

```
string City;
```

```
string Country;
```

```
float MonthlySalary;
```

```
char Gender;
```

```
bool isMarried;
```

 The user will provide the values later when the program runs.

- ◆ First, the program asks the user to enter their name:

```
cout << "Please enter your Name:" << endl;
```

```
getline(cin, Name);
```

Steps in main()

1. Declare the variables

- ◆ Create the variables that will store the user's input.

2. Ask for the Name

- ◆ Use `getline()` to read the user's full name.

3. Ask for the Age

- ◆ Use `cin` to read the user's age.

4. Use `cin.ignore()`

- ◆ Clear the leftover newline before using `getline()` again.

5. Ask for City and Country

- ◆ Use `getline()` to read text values.

6. Ask for Monthly Salary and Gender

- ◆ Use `cin` to read the numeric and character values.

7. Ask for Married Status

- ◆ Use `cin` to read 1 or 0

8. Calculate Yearly Salary

- ◆ Multiply the monthly salary by 12

9. Display the Results

- ◆ Print all the collected information to the screen.

💡 Example

```
cout << "Please enter your Name:" << endl;  
getline(cin, Name);  
cout << "Please enter your Age:" << endl;  
cin >> Age;  
cin.ignore();
```

➡ Results

```
Please enter your Name:  
Sara  
Please enter your Age:  
43  
Name: Sara  
Age: 43
```

- ◆ After using `cin >> Age`, the program uses:
`cin.ignore();`
- ◆ `cin.ignore()` removes the leftover newline character from the input buffer.
- ◆ This is important because the next input uses `getline()`

Note

- ◆ `getline()` is useful when the input can contain spaces.
- ◆ Without `cin.ignore()`, the following `getline()` may read the leftover newline instead of waiting for the user's input.

⊘ Warning

- ◆ When you use `cin >>` followed by `getline()`, you may need:
`cin.ignore();`
- ◆ Otherwise, `getline()` can immediately read the leftover newline character.



Comparison

▨ cin >>

- ✓ Reads numbers, characters, and simple input.
- ✓ Stops reading a string at whitespace.
- ✓ Example: `cin >> Age;`

▤ getline()

- ✓ Reads a complete line.
- ✓ Can read spaces.
- ✓ Example: `getline(cin, Name);`

✦✦ Summary

✓ User input allows a program to receive values from the user while it is running.

- `cin >>` → reads input.

- `getline()` → reads a complete line of text.

- `cin.ignore()` → removes the leftover newline character.

- `cin >> isMarried` → reads `1` or `0` into a Boolean variable.

- `? :` → chooses between two values based on a condition.

`cin >> variable;`

or

`getline(cin, variable);`

Thank you 😊